



**WYDZIAŁ FIZYKI
I INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Maciej Bujalski

Kierunek: informatyka

Specjalność: informatyka stosowana

Specjalizacja: aplikacje mobilne

Numer albumu: 386012

Rotacyjnie inwariantne sieci neuronowe

Praca magisterska

wykonana pod kierunkiem

dr Krzysztofa Podlaskiego

w Katedrze Systemów Inteligentnych, WFiIS UŁ

Łódź 2025

Spis treści

Wstęp	3
Cel i motywacja pracy	3
Opis pracy	4
Zakres tematyczny	5
Ujęte w zakresie	5
Poza zakresem	5
Artefakty pracy	6
Organizacja pracy	6
Podstawy teoretyczne	7
Wprowadzenie do sieci konwolucyjnych (CNN)	7
Operacja splotu	7
Receptywne pole	7
Nieliniowości i normalizacja	8
Pooling i głowa klasyfikacyjna	8
Regularizacja i trening	8
Triki architektoniczne	8
Ekwawariancja translacyjna (kontrast do rotacyjnej)	8
Inwariancja translacyjna i rotacyjna	9
Problemy z rotacyjną inwariancją w klasycznych CNN	9
Przegląd literatury (E(2)-Equivariant CNNs, CyCNN)	9
Opis zbiorów danych	10
MNIST (cyfry odręczne)	10
GTSRB Gray (znaki drogowe w odcienach szarości)	10
GTSRB RGB (znaki drogowe)	10
LEGO (obiekty 3d rzutowane na 2d)	10
Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów	10
Architektury modeli	10
Standardowe CNN	10
Rotacyjnie inwariantne sieci (np. CyResNet, CyVGG, G-CNN)	10
Transformacje polarne: linearpolar vs logpolar	10
Implementacja i środowisko eksperymentalne	10
Python, PyTorch	10
Struktura projektu	10
Automatyzacja: skrypty trenowania, testowania, ewaluacji	10
Obsługa GPU, Docker, WSL	10
Organizacja logów, modeli, confusion matrixów	10
Eksperymenty	11
Scenariusze trenowania/testowania (opis JSON)	11
Pomiar skuteczności (accuracy, macierze pomyłek)	11

Śledzenie metryk: średnia, mediana, odchylenie standardowe	11
Analiza skuteczności względem rotacji	11
Ranking modeli	11
Porównanie wyników	11
CyCNN vs klasyczne CNN	11
VGG vs CyVGG	11
Resnet vs CyResNet	11
CyVGG vs CyResNet	11
Wpływ transformacji (linearpolar vs logpolar)	11
Wydajność na różnych zbiorach	11
Wnioski	11
Skuteczność rotacyjnych architektur	11
Wnioski z automatyzacji i systematyzacji ewaluacji	11
Propozycje dalszych badań	11
Aneks	11
Listingi kodów	11
Dodatkowe wykresy, tablice wyników	11
Bibliografia	12

Wstęp

Obrazy otaczają nas z każdej strony: od zdjęć ze smartfonów, zdjęcia satelitarne, przez monitoring miejski, katalogi produktów i systemy kontroli jakości na liniach produkcyjnych, po systemy wspomagania jazdy. Choć współczesne modele rozpoznawania obrazu radzą sobie bardzo dobrze, w praktyce bywają wrażliwe na pozornie drobne zmiany takie jak obrócenie obiektu o kilkanaście stopni czy niewielki przechył kamery. To, co dla człowieka jest naturalne i natychmiast rozpoznawalne (znak drogowy pod kątem, cyfra obrócona na kartce), dla klasycznej konwolucyjnej sieci neuronowej bywa problemem. Rdzeń trudności to brak naturalnej inwariantności względem rotacji: standardowe CNN-y „z definicji” lepiej radzą sobie z przesunięciami niż z obrotami [1], [2].

W ostatnich latach pojawiło się kilka dróg domknięcia tej luki. Jedną to poszerzanie danych o zrotowane przykłady - poprawia odporność, ale wydłuża trening i nie gwarantuje uogólnienia na wszystkie kąty. Druga to architektury z wbudowaną geometrią: sieci grupowo równoważne (G-CNN, E(2)-equivariant) [3], [4], sieci cykliczne (CyCNN; w szczególności **CyVGG** i **CyResNet**) operujące na wielu orientacjach oraz przekształcenia do układów polarnych (linear-polar i log-polar), które „prostują” rotacje do przesunięć. Cel jest wspólny: by model rozpoznawał „to samo” niezależnie od orientacji, bez agresywnego dublowania danych.

Niniejsza praca skupia się na praktycznej weryfikacji tych podejść. Przygotowano zbiory obejmujące m.in. odręcznie napisane cyfry, znaki drogowe (w kolorze i w odcieniach szarości) oraz syntetyczne obiekty 3D rzutowane na 2D (klocki LEGO), a następnie rozszerzono je o kontrolowane rotacje. Zaimplementowano i porównano wybrane architektury rotacyjnie inwariantne i ich warianty bazowe w **PyTorchu** [5], mierząc wpływ transformacji (linear-polar vs. log-polar), wyboru architektury i zakresu kątów na jakość predykcji. Obliczenia realizowano na kartach: **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co skróciło czas trenowania i umożliwiło szeroki przegląd eksperymentów; środowisko uruchomieniowe ustandaryzowano z użyciem **Dockera** dla powtarzalności.

Celem pracy jest nie tylko pokazanie, że „da się” uzyskać odporność na rotacje, ale przede wszystkim wskazanie, **kiedy** i **jakim kosztem** ją osiągamy oraz które techniki przynoszą największy zysk względem klasycznych CNN-ów, jak wpływają na stabilność i szybkość uczenia, a także które konfiguracje są najpraktyczniejsze w realnych zastosowaniach (OCR, rozpoznawanie znaków, analiza obiektów technicznych). W dalszej części pracy przedstawiono podstawy, dane i augmentację, architektury, środowisko eksperymentalne, protokoły ewaluacji oraz wyniki z analizą i wnioskami.

Cel i motywacja pracy

Konwolucyjne sieci neuronowe (CNN) charakteryzują się zdolnością do analizy obrazów z zachowaniem niezmienniczości względem translacji. Jednak wciąż brakuje powszechnie uznanych architektur, które zapewniałyby inwariantność względem rotacji. Celem niniejszej pracy jest analiza skuteczności rozwiązań zaproponowanych w literaturze, ukierunkowanych na odporność modeli na rotację danych wejściowych (np. CyCNN [4]).

W ramach pracy zostały przygotowane wzbogacone zbiory danych, obejmujące m.in. zdjęcia odręcznie napisanych liter, znaki drogowe (zarówno w kolorze, jak i w odcieniach szarości) oraz obiekty 3D rzutowane na 2D (klocki LEGO), rozszerzone o kontrolowane rotacje obrazów.

Na podstawie tych zbiorów została przeprowadzona implementacja i ewaluacja wybranych architektur rotacyjnie inwariantnych z wykorzystaniem **PyTorch**. Obliczenia zostały wykonane przy użyciu kart graficznych **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co umożliwiło przyspieszenie trenowania. Otrzymane wyniki zostały porównane z rezultatami klasycznych sieci konwolucyjnych w celu oceny realnych korzyści z rozwiązań inwariantnych względem rotacji.

Opis pracy

Praca magisterska wykorzystuje zaawansowane technologie i narzędzia wspierające badania nad rotacyjnie inwariantnymi sieciami neuronowymi oraz ich zastosowaniem w przetwarzaniu obrazów. W realizacji projektu zastosowano następujące rozwiązania technologiczne:

- **Język programowania Python** - podstawowe narzędzie do implementacji algorytmów oraz obsługi frameworków uczenia maszynowego, dzięki wszechstronności i ekosystemowi bibliotek [6].

Frameworki uczenia maszynowego:

- **PyTorch** - elastyczny framework do szybkiego prototypowania i trenowania modeli [7].
- **TensorFlow** - narzędzie do budowy, trenowania i wdrażania modeli ML/DL [8].
- **Modele cykliczne (CyCNN)**. W pracy zostało przyjęte podejście, w którym obraz został przemapowany do współrzędnych (ρ, φ) . Dzięki temu obrót R_α staje się przesunięciem o α po osi φ . Konwolucje zostały zastąpione warstwami cylindrycznymi (CyConv) z cyklicznym paddingiem po φ . Dla każdego filtra zostało przygotowanych n orientacji; odpowiedzi zostały złożone z dodatkową osią „orientacja”. Obrót wejścia powoduje cykliczny shift po tej osi, a pooling po orientacjach daje inwariancję względem rotacji. Został ustawiony stały środek układu polarnych, stała siatka próbkowania oraz biliniarna interpolacja; padding po φ został ustawiony na cykliczny. Implementacja została wykonana w **PyTorchu** (punkt odniesienia: CyCNN [4]).
- **Wykorzystanie akceleracji GPU (NVIDIA)** - obliczenia zostały znacząco przyspieszone dzięki użyciu kart **RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**. Frameworki wspierają **CUDA** oraz **cuDNN**, co umożliwia efektywne wykorzystanie zasobów GPU [9], [10].
- **Konteneryzacja za pomocą Dockera** - odizolowane środowiska uruchomieniowe ułatwiły replikację i współdzielenie projektu, także z obsługą GPU [11].

Zakres tematyczny

Niniejsza praca dotyczy odporności modeli klasyfikacji obrazów na rotacje w płaszczyźnie. Skupiono się na porównaniu klasycznych architektur z ich wersjami rotacyjnie inwariantnymi oraz na wpływie przekształceń polarnych na jakość predykcji. Badania zostały przeprowadzone na obrazach 2D i rotacjach planarnych. W szczególności zostały porównane warianty bazowe **VGG/ResNet** z wersjami cyklicznymi **CyVGG/CyResNet**, a także wpływ mapowania **linear-polar** i **log-polar**. Eksperymenty zostały wykonane na zbiorach **MNIST**, **GTSRB_gray**, **GTSRB_RGB** i **LEGO** z kontrolowanymi rotacjami.

Ujęte w zakresie

- **Architektury modeli:** zostały zaimplementowane i porównane warianty bazowe **VGG** oraz **ResNet** [12], [13], a także ich wersje cykliczne **CyVGG** i **CyResNet** (modele rotacyjnie inwariantne) [4].
- **Przekształcenia polarne:** została oceniona użyteczność mapowania **linear-polar** oraz **log-polar** jako etapów wstępnego przetwarzania służących „prostowaniu” rotacji do przesunięć [4], [14].
- **Zbiory danych:** zostały wykorzystane następujące zbiory: **MNIST** (odręczne cyfry, 28x28 → 32x32, grayscale) [15], **LEGO** (syntetyczne obiekty 3D rzutowane na 2D, 96x96, grayscale), **GTSRB** (znaki drogowe, 32x32, grayscale) oraz **GTSRB_RGB** (wersja kolorowa) [16]. Zbiory zostały rozszerzone o kontrolowane rotacje oraz zostały przygotowane spójne podziały zbiorów na train/val/test.
- **Augmentacja i protokół:** zostały zdefiniowane zakresy kątów, liczba powtórzeń i podział na zbiory train/val/test (uczący/ walidacyjny/testowy), z możliwością powtórzeń trenowań.
- **Środowisko i implementacja:** został wykorzystany **PyTorch** z akceleracją **CUDA/cuDNN** na kartach **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **NVIDIA GeForce RTX 3060 12 GB** [7], [9], [10]. Przygotowane zostały skrypty w Pythonie do trenowania, testowania i ewaluacji [6]. Środowisko uruchomieniowe zostało ustandaryzowane z użyciem **Dockera** [11].
- **Metryki i analiza:** została przeprowadzona ocena jakości (accuracy, macierze pomyłek), analiza stabilności (średnia/mediana/odchylenie standardowe), wpływ kąta rotacji na skuteczność oraz koszt obliczeniowy (czas trenowania, rozmiar modelu).

Poza zakresem

- Detekcja obiektów i segmentacja — w pracy rozpatrywana jest wyłącznie klasyfikacja [17], [18].
- Inwariancja względem skali, ścinania i pełnych przekształceń afinicznych — analizowana jest tylko rotacja w płaszczyźnie [19], [20].
- Rotacje w geometrii 3D oraz zagadnienia widzenia stereo — pozostają poza zakresem [21], [22].

- Trening na bardzo dużych korpusach z pre-treningiem self-supervised oraz szerokim AutoML/hyper-search — nie został realizowany [23], [24], [25].
- Odporność na silne zakłócenia (szum, okluzje) — poza zakresem; skupiono się wyłącznie na rotacji [26], [27].

Artefakty pracy

- Repozytoria z kodem, skryptami i plikami konfiguracyjnymi (PyTorch).
- Pliki z konfiguracjami eksperymentów i opisem danych.
- Wytrenowane wagi modeli (wybrane checkpointy) oraz raporty z ewaluacji.
- Tekst pracy z dokumentacją eksperymentów i wnioskami.

Organizacja pracy

Struktura pracy została ułożona tak, by od podstaw przejść do wyników. W rozdziale **Podstawy teoretyczne** zostały zebrane pojęcia i narzędzia: CNN, inwariancja/ekwawariancja, przekształcenia polarne oraz prace pokrewne (G-CNN, CyCNN). W **Opisie zbiorów danych** zostały przedstawione MNIST, LEGO, GTSRB (gray) i GTSRB_RGB oraz sposób augmentacji (rotacje, podziały train/val/test). W **Architekturach modeli** zostały opisane warianty bazowe **VGG/ResNet** oraz wersje cykliczne **CyVGG/CyResNet**, wraz z transformacjami linear-polar / log-polar. Rozdział **Implementacja i środowisko** zawiera szczegóły techniczne: **PyTorch**, **CUDA** oraz **Tensor** (RTX 3070TI 8GB, RTX 3060 12 GB), **Docker**, strukturę projektu i skrypty. W **Eksperymentach** zostały zdefiniowane scenariusze, metryki i sposób ewaluacji. Dalej, w **Porównaniu wyników**, zostały zestawione modele (VGG vs CyVGG, ResNet vs CyResNet, wpływ transformacji) i omówiona stabilność/czas. Na końcu **Wnioski** zbierają najważniejsze obserwacje i wskazują kierunki dalszych badań; **Aneks** zawiera kody i dodatkowe wykresy.

Podstawy teoretyczne

Wprowadzenie do sieci konwolucyjnych (CNN)

CNN zostały zaprojektowane pod dane siatkowe (obrazy 2D). Kluczowe cechy to **lokalne receptywne pola**, **współdzielenie wag** oraz **splot**, co daje skalowalność na duże obrazy i lepsze uogólnianie niż pełne połączenia. Zamiast patrzeć na cały kadr naraz, ten sam filtr „przesuwa się” po lokalnych oknach i uczy detektorów krawędzi, tekstur i kształtów, a wyżej - bardziej złożonych cech. [1], [28] Dla przesunięcia $\mathcal{T}_t$ i jądra K mamy ekwawariancję translacyjną:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K).$$

Czyli: jak przesuniemy obraz, to mapa cech również przesuwa się o ten sam wektor. Inwariancję na przesunięcia „domyka się” w praktyce poolingiem (lokalnym/globalnym) albo stride’em - rzadsze próbkowanie sprawia, że decyzja nie zależy od dokładnej pozycji obiektu. [1], [2]

Operacja splotu

Dla wejścia $X \in \mathbb{R}^{C_{in} \times H \times W}$ i filtra $K \in \mathbb{R}^{C_{in} \times k \times k}$ wyjście kanału c jest definiowane jako:

$$Y_c(u, v) = \sum_{i=1}^{C_{in}} \sum_{a,b} K_{c,i}(a, b) X_i(u - a, v - b).$$

W praktyce stosowany jest **cross-correlation** (bez odwrócenia kernela), ale nazwa „conv” została.

Parametry warstwy nie zależą od $(u, v) \rightarrow$ **współdzielenie wag**. Lokalny zasięg $k \times k \rightarrow$ **złożoność nie rośnie** z wysokością/szerokością obrazu jak w warstwach w pełni połączonych.

Stride, padding, rozmiary Dla wysokości:

$$H_{out} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1.$$

- **padding** p - kontroluje utratę informacji na brzegach (same/valid), - **stride** s - realizuje podsampling wprost w splotach, - **dilacja** - powiększa efektywne pole widzenia bez zwiększania liczby parametrów.

Receptywne pole

Efektywne receptywne pole rośnie z głębokością. Dla warstw z kernelami k_ℓ i stride’ami s_ℓ :

$$R_1 = k_1, \quad R_\ell = R_{\ell-1} + (k_\ell - 1) \prod_{j < \ell} s_j.$$

Projekt architektury został dobrany tak, by R_ℓ obejmowało obiekty istotne dla klasyfikacji.

Nieliniowości i normalizacja

- **Aktywacje:** zostały użyte funkcje typu ReLU/LeakyReLU (nieliniowość, brak problemu zaniku dla dodatnich wartości).
- **Batch Normalization:** stabilizuje statystyki w mini-batchu, przyspiesza uczenie:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta.$$

- Alternatywnie **Layer/GroupNorm** - niezależne od rozmiaru batcha.

Pooling i głowa klasyfikacyjna

- **Max/avg-pooling** zostały użyte do redukcji rozmiaru i uzyskania częściowej inwariancji translacyjnej.
- **Global Average Pooling (GAP)** agreguje mapy cech do wektora (mniej parametrów niż pełne połączenia).
- Ostatnia warstwa to **softmax**; dla klasy c :

$$p(c|x) = \frac{e^{z_c}}{\sum_j e^{z_j}}, \quad \mathcal{L} = -\log p(y|x) \text{ (cross-entropy)}.$$

Regularizacja i trening

- **Weight decay (L2), dropout** (tam gdzie zasadne) oraz **augmentacja** danych zostały użyte do ograniczenia przeuczenia.
- Optymalizacja: **SGD/Adam** z harmonogramem uczenia; ziarna losowe zostały ustawione dla powtarzalności.

Triki architektoniczne

- **1×1 conv** (bottleneck) - mieszanie kanałów, redukcja kosztu.
- **Głębokie stosy z residualami** (jak w ResNet) ułatwiają przepływ gradientu.
- **Depthwise separable conv** (opcjonalnie) - rozdzielenie splotu po kanałach i miksowanie punktowe dla redukcji FLOPs.

Ekwawariancja translacyjna (kontrast do rotacyjnej)

Dla operatora przesunięcia $\mathcal{T}_t$ i splotu „conv” zachodzi:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K),$$

co wyjaśnia, dlaczego klasyczne CNN dobrze radzą sobie z przesunięciami. Brak analogicznego, wbudowanego mechanizmu dla rotacji motywuje użycie przekształceń polarnych i/lub modeli cyklicznych w dalszych rozdziałach.

Inwariancja translacyjna i rotacyjna

Niech $\mathcal{R}_\alpha$ oznacza obrót o kąt α , a Φ - mapę cech.

Ekwawariancja:

$$\Phi(\mathcal{R}_\alpha X) = \Pi_\alpha \Phi(X)$$

Inwariancja:

$$\Phi(\mathcal{R}_\alpha X) = \Phi(X)$$

W praktyce inwariancja rotacyjna została uzyskiwana przez: (a) augmentację rotacją, (b) architekturę śledzącą orientację (oś „kąt”), (c) przemapping do polarnych, gdzie obrót staje się przesunięciem po osi φ .

Problemy z rotacyjną inwariancją w klasycznych CNN

- Filtry są kierunkowe - jeden kernel nie pokrywa wielu orientacji.
- Sama augmentacja rotacją wydłuża trening i zostawia „dziury” między kątami.
- Interpolacja przy rotacjach generuje aliasing i artefakty na brzegach.
- Padding zero/same łamie symetrię przy krawędziach.
- Brakuje jawnej osi „orientacja” (sieć nie „wie”, pod jakim kątem aktywacja została wykryta).

Przegląd literatury (E(2)-Equivariant CNNs, CyCNN)

CyCNN (użyte w pracy). Zostało przyjęte mapowanie $(x, y) \mapsto (\rho, \varphi)$ i warstwy cylindryczne (CyConv) z cyklicznym paddingiem po osi φ . Dla każdego filtra zostało przygotowanych n wariantów obróconych o $\theta_k = 2\pi k/n$ (grupa C_n); odpowiedzi zostały złożone z dodatkową osią „orientacja”. Obrót wejścia $R_k \in C_n$ daje cykliczny shift po tej osi (**ekwawariancja**), a pooling po orientacjach daje **inwariancję**. W pracy zostały użyte CyVGG i CyResNet.

E(2)-equivariant / steerable CNNs (kontekst). W literaturze zostały opisane sploty grupowe i sterowalne jądra zapewniające ekwawariancję względem translacji i rotacji w E(2) (ciągłe kąty). Wymagają projektowania jąder z użyciem reprezentacji grupy i zwykle wyższego kosztu obliczeń. W tej pracy traktowane jako tło teoretyczne.

Opis zbiorów danych

MNIST (cyfry odręczne)

GTSRB Gray (znaki drogowe w odcieniach szarości)

GTSRB RGB (znaki drogowe)

LEGO (obiekty 3d rzutowane na 2d)

Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów

Architektury modeli

Standardowe CNN

Rotacyjnie inwariantne sieci (np. CyResNet, CyVGG, G-CNN)

Transformacje polarne: linearpolar vs logpolar

Implementacja i środowisko eksperymentalne

Python, PyTorch

Struktura projektu

Automatyzacja: skrypty trenowania, testowania, ewaluacji

Obsługa GPU, Docker, WSL

Organizacja logów, modeli, confusion matrixów

Eksperymenty

Scenariusze trenowania/testowania (opis JSON)

Pomiar skuteczności (accuracy, macierze pomyłek)

Śledzenie metryk: średnia, mediana, odchylenie standardowe

Analiza skuteczności względem rotacji

Ranking modeli

Porównanie wyników

CyCNN vs klasyczne CNN

VGG vs CyVGG

Resnet vs CyResNet

CyVGG vs CyResNet

[...] cyresnet56 uczył się dłużej niż cyvgg19, ale dawał bardziej stabilne wyniki, w szczególności w przypadku funkcji aktywacji typu logpolar. I jak to się mówi - nie można zjeść ciastka i mieć go też (ang. „You can’t eat your cake and have it too” [29]).

Wpływ transformacji (linearpolar vs logpolar)

Wydajność na różnych zbiorach

Wnioski

Skuteczność rotacyjnych architektur

Wnioski z automatyzacji i systematyzacji ewaluacji

Propozycje dalszych badań

Aneks

Listingi kodów

Dodatkowe wykresy, tablice wyników

Bibliografia

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: <https://www.deeplearningbook.org/>
- [2] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning.” 2016. Available: <https://arxiv.org/abs/1603.07285>
- [3] T. Cohen and M. Welling, “Group equivariant convolutional networks,” in *Proceedings of the 33rd international conference on machine learning (ICML)*, 2016. Available: <https://proceedings.mlr.press/v48/cohen16.html>
- [4] J. Kim, W. Jung, H. Kim, and J. Lee, “CyCNN: A rotation invariant CNN using polar mapping and cylindrical convolution layers,” *arXiv preprint arXiv:2007.10588*, 2020, Available: <https://arxiv.org/abs/2007.10588>
- [5] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in neural information processing systems 32 (NeurIPS)*, 2019. Available: <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [6] Python Software Foundation, “Python 3 documentation.” 2025. Available: <https://docs.python.org/3/>
- [7] PyTorch Core Team, “PyTorch documentation (stable).” 2025. Available: <https://pytorch.org/docs/stable/>
- [8] TensorFlow Authors, “TensorFlow API documentation.” 2025. Available: https://www.tensorflow.org/api_docs
- [9] NVIDIA Corporation, “CUDA toolkit documentation.” 2025. Available: <https://docs.nvidia.com/cuda/>
- [10] NVIDIA Corporation, “NVIDIA cuDNN developer guide.” 2025. Available: <https://docs.nvidia.com/deeplearning/cudnn/developer-guide/index.html>
- [11] Docker, Inc., “Docker documentation.” 2025. Available: <https://docs.docker.com/>
- [12] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014, Available: <https://arxiv.org/abs/1409.1556>
- [13] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016, pp. 770–778. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
- [14] B. S. Reddy and B. N. Chatterji, “An FFT-based technique for translation, rotation and scale-invariant image registration,” *IEEE Transactions on Image Processing*, vol. 5, no. 8, pp. 1266–1271, 1996, doi: 10.1109/83.506761.
- [15] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST database of handwritten digits.” 1998. Available: <http://yann.lecun.com/exdb/mnist/>
- [16] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “The german traffic sign recognition benchmark: A multi-class classification competition,” in *Proceedings of the international joint conference on neural networks (IJCNN)*, 2011, pp. 1453–1460. doi: 10.1109/IJCNN.2011.6033395.

- [17] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask r-CNN,” in *Proceedings of the IEEE international conference on computer vision (ICCV)*, 2017. Available: https://openaccess.thecvf.com/content_ICCV_2017/papers/He_Mask_R-CNN_ICCV_2017_paper.pdf
- [18] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *MICCAI*, in LNCS, vol. 9351. 2015, pp. 234–241. Available: https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28
- [19] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, Available: <https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>
- [20] M. Jaderberg, K. Simonyan, A. Zisserman, and K. Kavukcuoglu, “Spatial transformer networks,” in *Advances in neural information processing systems (NIPS)*, 2015. Available: <https://proceedings.neurips.cc/paper/2015/file/33ceb07bf4eeb3da587e268d663aba1a-Paper.pdf>
- [21] C. Esteves, C. Allen-Blanchette, A. Makadia, and K. Daniilidis, “Learning SO(3)-equivariant representations with spherical CNNs,” in *Proceedings of the european conference on computer vision (ECCV)*, 2018, pp. 52–68. Available: https://openaccess.thecvf.com/content_ECCV_2018/html/Carlos_Esteves_Learning_SO3_Equivariant_ECCV_2018_paper.html
- [22] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*, 2nd ed. Cambridge University Press, 2004.
- [23] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proceedings of the 37th international conference on machine learning (ICML)*, PMLR, 2020. Available: <https://proceedings.mlr.press/v119/chen20j/chen20j.pdf>
- [24] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2020. Available: https://openaccess.thecvf.com/content_CVPR_2020/papers/He_Momentum_Contrast_for_Unsupervised_Visual_Representation_Learning_CVPR_2020_paper.pdf
- [25] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: Bandit-based configuration evaluation for hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018, Available: <https://jmlr.org/papers/volume18/16-558/16-558.pdf>
- [26] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *International conference on learning representations (ICLR)*, 2019. Available: <https://web.engr.oregonstate.edu/~tgd/publications/hendrycks-dietterich-benchmarking-neural-network-robustness-to-common-corruptions-and-perturbations-iclr2019.pdf>
- [27] T. DeVries and G. W. Taylor, “Improved regularization of convolutional neural networks with cutout.” 2017. Available: <https://arxiv.org/abs/1708.04552>
- [28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, 1998.

- [29] T. J. Kaczynski, "Industrial society and its future," *The Washington Post*, Sep. 1995, Available: <https://www.washingtonpost.com/wp-srv/national/longterm/unabomber/manifesto.text.htm>